

Algorithmic Effects of Prompt Engineering Techniques on Large Language Model Output

1st Sarwagya Acharya

DEPARTMENT

ORGANIZATION

CITY, COUNTRY

EMAIL SARWAGYA

2nd Nitesh Pant

DEPARTMENT

ORGANIZATION

CITY, COUNTRY

EMAIL NITESH

3rd Brishav Joshi

DEPARTMENT

ORGANIZATION

CITY, COUNTRY

EMAIL BRISHAV

Abstract—Prompt engineering has rapidly evolved from ad-hoc text tweaks into a family of techniques that systematically reshape how large language models (LLMs) generate output. Yet the literature largely treats these techniques as linguistic patterns, leaving their algorithmic consequences under-quantified. This paper analyses prompt engineering techniques through the lens of the conditional distribution that an LLM defines over output tokens, and characterises how each technique shifts that distribution via prompting, decoding, and reasoning-trace construction. We taxonomise the major technique families, formalise their effect on sampling, attention, and output diversity, and report a controlled empirical comparison across representative tasks. The results show that the algorithmic effect of a prompt often dominates its lexical content, and we recommend evaluation protocols that make this effect measurable.

Index Terms—prompt engineering, large language models, chain-of-thought, decoding strategies, evaluation, algorithmic analysis

I. INTRODUCTION

Large language models (LLMs) are now routinely deployed for question answering, code synthesis, summarisation, and tool use, and the practical quality of an LLM application is shaped as much by the prompt that surrounds a model call as by the model weights themselves [1], [2]. A growing catalogue of prompt engineering techniques—zero- and few-shot exemplars, chain-of-thought (CoT) [3], self-consistency [4], tree-of-thoughts (ToT) [5], ReAct [6], and retrieval-augmented prompts [7]—reports substantial gains on established benchmarks, often with no change to the underlying model.

The dominant framing in this literature is empirical and qualitative: a technique is described by its prompting template and illustrated with selected outputs. While useful, this framing obscures a more fundamental question, namely *what does the technique change inside the model?* In this paper we argue that the principal techniques are best understood as algorithmic operations on the conditional distribution $p_\theta(y | P, x)$ that an LLM defines over output tokens. Under this view, a prompt is not merely a textual wrapper; it is a control input that perturbs sampling, restructures attention, alters logit geometry, and conditions intermediate computational traces. The size of the resulting behavioural change often rivals, and sometimes exceeds, the difference between adjacent model scales.

The contributions of this paper are as follows.

- A taxonomy that organises prompt engineering techniques by the algorithmic object they modify: the prompt template, the decoding policy, or the reasoning trace.
- A formal framework that expresses each technique family as a transformation on $p_\theta(y | P, x)$ and on the underlying arg max or sampling operator.
- A controlled empirical study that isolates the algorithmic effect of prompting from the lexical effect by holding the prompt template fixed and varying decoding parameters (T, k, p) , and vice versa.
- An evaluation protocol that we recommend for future prompt engineering research, designed to make algorithmic and lexical contributions separable.

The remainder of this paper is organised as follows. Section II reviews prior surveys and the foundational techniques that motivate this work. Section III introduces the taxonomy and notation. Section IV formalises the algorithmic effect of each technique. Section V describes the experimental design. Section VI reports the results, and Sections VII–IX discuss them, list limitations, and conclude.

II. BACKGROUND AND RELATED WORK

The prompt engineering literature spans prompting patterns, decoding-time methods, and reasoning-trace construction. We organise the review along the same three axes used in our taxonomy (Section III).

A. Prompting patterns as control inputs

Brown et al. [1] demonstrated that scaling model size produces in-context learning, in which a small set of exemplars in the prompt elicits task behaviour without weight updates. The result established prompting as a first-class interface to LLMs and motivated subsequent work on exemplar selection, ordering, and formatting. Min et al. [2] further showed that the ground-truth labels in exemplars contribute less than the input-output format, evidence that prompts act partly as structural scaffolds for the model’s own computations. Two findings sharpen this picture. Lu et al. [10] report that permuting a fixed exemplar set moves accuracy across a range wide enough to swamp the difference between competing methods, and Zhao et al. [11] trace part of that sensitivity to a prior bias in the label distribution that can be corrected without touching the

wording of the prompt. Both results are difficult to explain if a prompt is regarded purely as a piece of natural language; both are unsurprising if it is regarded as a control input.

B. Reasoning-trace construction

Wei et al. [3] introduced chain-of-thought prompting, appending intermediate reasoning steps to exemplars and observing large gains on arithmetic, commonsense, and symbolic tasks. The technique reframes generation as a two-stage process: sample a reasoning chain r , then sample the answer y conditioned on r . Kojima et al. [9] showed that a single trigger phrase reproduces much of the effect without exemplars, which separates the contribution of the trace from that of the demonstrations. Wang et al. [4] extended CoT with self-consistency, sampling multiple chains and selecting the most frequent final answer; the gain over greedy CoT demonstrates that the underlying distribution is multimodal and that marginalising over reasoning paths reallocates probability mass towards correct answers. Yao et al. [5] generalised chains to trees (tree-of-thoughts), Yao et al. [6] interleaved reasoning with tool calls in ReAct, and Madaan et al. [12] closed the loop by feeding the model’s own critique back into the context. Across these works, the common algorithmic move is to expose intermediate computation to the sampling operator.

C. Decoding-time and retrieval augmentation

Beyond reasoning traces, decoding-time methods such as temperature, top- k [13], and nucleus (top- p) sampling [8] directly parametrise the output distribution; Meister et al. [14] argue that the useful criterion is closeness to the expected information content rather than raw probability mass. Retrieval-augmented generation (RAG) [7] couples the LLM to an external retriever, conditioning generation on documents fetched at inference time. While RAG is often presented as a knowledge-injection technique, it is, in our framing, a prompting technique: it changes the conditioning context P of $p_\theta(y | P, x)$ and thereby changes the posterior over outputs.

D. Positioning of this paper

Existing surveys [15], [16] catalogue prompting techniques and, in the case of automated prompt search [17], optimise over them, but they do not formalise the algorithmic effect of the resulting prompts. Empirical comparisons exist for individual techniques, yet they typically vary the prompt while holding decoding fixed, which confounds the algorithmic and lexical contributions. To our knowledge, no prior work isolates the algorithmic effect of prompting through controlled ablations on both the prompt template and the decoding policy. This paper fills that gap by treating prompting and decoding as two parameters of the same sampling operator and measuring each contribution independently.

E. Common notation and primer

Throughout the paper we write P for a prompt template, x for the user input, and y for the model output. An LLM with

parameters θ defines a conditional distribution $p_\theta(y | P, x)$. Decoding is parameterised by a temperature T , a top- k cutoff, and a nucleus threshold p . We treat CoT, self-consistency, and ToT as sampling procedures over reasoning traces r followed by answers y , and we treat retrieval augmentation as a stochastic rewrite of P . These objects are formalised in Section IV.

III. TAXONOMY OF PROMPT ENGINEERING TECHNIQUES

A technique that improves a benchmark score tells us little by itself, because the improvement may originate at any of three points in the generation pipeline. We therefore classify techniques by the object they modify rather than by the surface form they take.

A. Three operator classes

Definition 1 (Prompting method). *A prompting method is a triple $M = (\Phi, \Psi, \Omega)$, where Φ is a prompt-space operator acting on the conditioning context, Ψ is a decoding-space operator acting on the per-step token distribution, and Ω is a trace-space operator acting on the set of intermediate sequences that are generated before the answer is emitted. Any component may be the identity.*

Definition 1 is deliberately coarse. Its purpose is to make the following claim checkable: two techniques that are described very differently in prose may occupy the same cell of the taxonomy, and two techniques presented as variants of one another may not. Table I assigns the principal techniques to cells and records the cost each incurs.

B. Prompt-space operators

Prompt-space operators rewrite the conditioning context and leave both the decoding policy and the generated trace untouched. Instruction rewriting, role conditioning, exemplar selection and ordering, output-format scaffolding, and retrieval injection all belong here. What distinguishes them from one another is the amount of context they add and whether the added content is deterministic. Retrieval is the outlier: because the retriever returns a different document set for different inputs, the operator is stochastic even when the template around it is fixed, and any variance it introduces will be attributed to the prompt unless the retrieved set is logged.

C. Decoding-space operators

Decoding-space operators leave the context untouched and reshape the per-step distribution before a token is drawn. Temperature scaling is a smooth reparametrisation; top- k and nucleus sampling are support restrictions; beam search replaces per-step sampling with an approximate search for a high-scoring sequence. These operators are the only ones in the taxonomy that can be applied without changing a single character of the prompt, which makes them the natural control condition for the experiments in Section V.

D. Trace-space operators

Trace-space operators change what the model generates before it commits to an answer. The simplest, zero-shot CoT, adds a trigger phrase and lets the model produce a chain; the most elaborate, tree-of-thoughts, maintains a frontier of partial traces and expands it under a value estimate. Self-consistency sits between the two: it does not alter the shape of an individual trace but draws several and aggregates them. We stress that trace-space operators are not free. Every one of them buys accuracy with tokens, forward passes, or both, and Table I records the exchange rate.

E. Composition and interaction

The three operator classes do not compose independently, and the failure of independence is itself informative. Self-consistency is the clearest case: it requires $T > 0$, since m greedy samples are m copies of one chain, so the trace-space operator is only defined relative to a decoding-space operator that supplies diversity. Conversely, few-shot CoT couples Φ and Ω because the exemplars carry the traces. We therefore formulate the two hypotheses that the experiments in Section V are designed to test:

- H1 Holding the algorithmic structure of a prompt fixed, paraphrasing its wording produces a smaller change in task accuracy than moving between cells of Table I.
- H2 The interaction between trace-space and decoding-space operators is significant, so reporting a trace-space technique without its decoding configuration underdetermines the result.

IV. ALGORITHMIC FRAMEWORK

A. Base distribution

Let $\mathcal{V}$ be the vocabulary and let $y = (y_1, \dots, y_{|y|}) \in \mathcal{V}^*$ be an output sequence. An autoregressive LLM factorises

$$p_\theta(y \mid P, x) = \prod_{t=1}^{|y|} p_\theta(y_t \mid y_{<t}, P, x), \quad (1)$$

where each factor is obtained from a logit vector $z_t \in \mathbb{R}^{|\mathcal{V}|}$ by

$$p_\theta(y_t = v \mid y_{<t}, P, x) = \frac{\exp(z_{t,v})}{\sum_{u \in \mathcal{V}} \exp(z_{t,u})}. \quad (2)$$

Equations (1) and (2) contain every quantity a prompting technique can touch. A technique either changes z_t through the conditioning arguments, changes the map from z_t to a sampled token, or changes which sequences are generated and how they are combined.

B. Prompt-space operators as logit displacement

Let $\Phi : P \mapsto P'$ and write $z_t(P)$ for the logits obtained under context P . The operator induces a displacement

$$\Delta_t(\Phi) = z_t(P') - z_t(P) \in \mathbb{R}^{|\mathcal{V}|}. \quad (3)$$

Because the softmax in (2) is invariant to adding a constant to every logit, only the component of Δ_t orthogonal to $\mathbf{1}$ has any behavioural consequence. The effective displacement is therefore the centred vector

$$\hat{\Delta}_t(\Phi) = \Delta_t(\Phi) - \frac{1}{|\mathcal{V}|}(\mathbf{1}^\top \Delta_t(\Phi))\mathbf{1}, \quad (4)$$

and $\|\hat{\Delta}_t\|_2$ is a scalar summary of how hard the rewrite pushed on the model at step t . This distinction matters in practice: a prompt edit that raises every logit uniformly, for instance one that only lengthens the context without adding discriminative content, has $\hat{\Delta}_t \approx 0$ and cannot change behaviour no matter how many tokens it costs.

A second, coarser summary is the divergence induced at the distribution level,

$$D_\Phi = \mathbb{E}_x \left[\frac{1}{|y|} \sum_{t=1}^{|y|} D_{\text{KL}}(p_\theta(\cdot \mid y_{<t}, P', x) \| p_\theta(\cdot \mid y_{<t}, P, x)) \right], \quad (5)$$

which we report per token so that prompts of different lengths remain comparable. Attention offers a third view. Writing $\alpha_{t,S}^{(\ell,h)}$ for the mass that head h of layer ℓ places on a context span S at step t , the quantity $\sum_{\ell,h} \alpha_{t,S}^{(\ell,h)}$ measures how much of the model's read budget a newly inserted span attracts. A rewrite that raises D_Φ while leaving attention mass on the inserted span near zero is evidence of an indirect effect—a shift in position or in the boundary structure of the context—rather than of the model attending to the new text.

C. Decoding-space operators as measure transforms

Fix the logits and let Ψ act on them. Temperature scaling produces

$$q_T(v) = \frac{\exp(z_v/T)}{\sum_u \exp(z_u/T)}, \quad (6)$$

a Gibbs family whose Shannon entropy $H(q_T)$ is non-decreasing in T , with q_T approaching a point mass on $\arg \max_v z_v$ as $T \rightarrow 0^+$ and the uniform distribution as $T \rightarrow \infty$. Temperature is thus a continuous interpolation between the maximisation and the uniform-exploration regimes, and the two limits bracket everything a sampler can do without additional information.

Truncation operators behave differently. Let π order $\mathcal{V}$ by decreasing probability. Top- k retains $S_k = \{\pi_1, \dots, \pi_k\}$ and nucleus sampling retains the smallest prefix whose mass reaches p ,

$$S_p = \{\pi_1, \dots, \pi_\kappa\}, \quad \kappa = \min \left\{ j : \sum_{i \leq j} q(\pi_i) \geq p \right\}, \quad (7)$$

after which the retained mass is renormalised. Both discard the tail, but they discard it in different ways, and the difference is the point.

Proposition 1 (Support and entropy bounds). *Let $q^{(k)}$ be the renormalisation of q onto S_k . Then $|\text{supp}(q^{(k)})| \leq k$ and $H(q^{(k)}) \leq \log k$, both independent of the context. For nucleus sampling the analogous bound is $H(q^{(p)}) \leq \log \kappa(x, t)$, where κ is a function of the step-level distribution and is therefore not fixed in advance.*

Proposition 1 explains an asymmetry that is often observed but rarely stated: top- k imposes a hard, context-independent ceiling on per-step diversity, whereas nucleus sampling lets the ceiling float with the model’s own confidence, contracting the support where the model is certain and widening it where it is not. Reporting a single top- p value without the resulting κ statistics therefore conveys much less information than it appears to.

D. Trace-space operators as marginalisation and search

Let r denote an intermediate trace. The answer distribution the model actually defines is the marginal

$$p_\theta(y \mid P, x) = \sum_r p_\theta(y \mid r, P, x) p_\theta(r \mid P, x). \quad (8)$$

Direct answering approximates $\arg \max_y$ of (8) without ever materialising r . Greedy CoT does something different: it materialises one trace and returns the answer attached to it, which approximates

$$\hat{y}_{\text{CoT}} = \arg \max_y \max_r p_\theta(y, r \mid P, x), \quad (9)$$

the joint mode rather than the marginal mode. The two need not coincide.

Proposition 2 (Mode mismatch). *There exist distributions for which $\arg \max_y \max_r p(y, r) \neq \arg \max_y \sum_r p(y, r)$.*

Proof. Take two answers y_1, y_2 and three traces. Let $p(y_1, r_1) = 0.30$ and $p(y_1, r_2) = p(y_1, r_3) = 0$, while $p(y_2, r_1) = p(y_2, r_2) = p(y_2, r_3) = 0.23\bar{3}$. The joint mode selects y_1 with 0.30, but the marginals are $p(y_1) = 0.30 < p(y_2) = 0.70$. ■

Proposition 2 is the algorithmic content of self-consistency. Drawing m traces and taking the plurality answer is a Monte Carlo estimator of the marginal mode, so the technique corrects a systematic bias in greedy CoT rather than merely averaging away noise. Its reliability follows from a standard concentration argument.

Proposition 3 (Vote concentration). *Let y^* maximise the marginal with mass q^* , let q' be the largest mass on any other answer, and let $\gamma = q^* - q' > 0$. For m i.i.d. traces, the plurality vote returns y^* with probability at least $1 - (|\mathcal{Y}| - 1) \exp(-m\gamma^2/2)$.*

Proof. For each competing answer apply Hoeffding’s inequality to the difference of the two empirical frequencies, which has mean γ and bounded increments, then take a union bound over the at most $|\mathcal{Y}| - 1$ competitors. ■

Two consequences follow. First, the useful range of m is governed by the margin γ , not by task difficulty as such: items with a small margin need many samples and items with a large

margin need very few, which is what makes adaptive sample-allocation schemes profitable. Second, self-consistency cannot repair a distribution whose marginal mode is wrong, since increasing m only sharpens convergence to y^* . Errors of that kind must be addressed in prompt space or trace space.

Tree-of-thoughts replaces sampling with search. With branching factor b , depth d , and a value estimate $v(\cdot)$ over partial traces, the method returns

$$\hat{y}_{\text{ToT}} = \arg \max_y \max_{r \in \mathcal{R}_{b,d}} v(r) p_\theta(y \mid r, P, x), \quad (10)$$

where $\mathcal{R}_{b,d}$ is the set of traces the search actually visits. Compared with (9), the model’s own trace probability has been partly replaced by an external value signal; ToT therefore inherits the quality of v , and a poorly calibrated v degrades it towards an expensive beam search. ReAct is the same substitution carried out by the environment instead of a value function: the trace interleaves actions with observations $o_{1:n}$, and the conditioning distribution becomes $p_\theta(y \mid P, x, o_{1:n})$ with o_i drawn from a tool rather than from θ . This is why ReAct reduces exactly those errors that arise from missing information, and why it leaves errors of reasoning largely intact.

E. Cost model

Let C count forward passes and N count generated tokens. Direct answering has $C = 1$ and $N = O(|y|)$. CoT has $C = 1$ and $N = O(|r| + |y|)$. Self-consistency multiplies both by m . ToT costs $C = O(bd)$ passes plus the value calls. Because $|r| \gg |y|$ on reasoning tasks, the dominant term is usually trace length rather than the number of passes, so accuracy per thousand generated tokens is a more faithful efficiency measure than accuracy per call. We report both in Section VI-D.

F. Separating algorithmic from lexical effects

The framework above suggests a direct measurement. Let Φ_{lex} range over paraphrases that preserve algorithmic structure—same exemplar count, same trace shape, same output format—and let Φ_{alg} range over changes of cell in Table I. Fitting a two-factor model to the accuracy A over these two factors and their interaction, we define the algorithmic effect ratio

$$\text{AER} = \frac{\sigma_{\text{alg}}^2}{\sigma_{\text{alg}}^2 + \sigma_{\text{lex}}^2}, \quad (11)$$

where each term is the variance component attributable to that factor. An AER near 1 means the technique’s benefit survives rewording; a value near 0.5 means half of a reported gain is a property of the particular sentence used to elicit it. Together with D_Φ from (5) and the κ statistics of Proposition 1, this gives a small set of quantities that a prompt engineering paper can report and a reader can compare.

TABLE I
TAXONOMY OF PROMPT ENGINEERING TECHNIQUES BY MODIFIED ALGORITHMIC OBJECT

Technique	Operator	Object modified	Forward passes	Added tokens	Stochastic?
Instruction rewriting	Φ	conditioning context	1	$O(I)$	no
Role / persona conditioning	Φ	conditioning context	1	$O(1)$	no
Few-shot exemplars	Φ	conditioning context	1	$O(n\bar{\ell})$	no
Format scaffolding	Φ	conditioning context	1	$O(1)$	no
Retrieval augmentation	Φ	conditioning context	1+retrieval	$O(n_d\bar{\ell}_d)$	yes
Temperature scaling	Ψ	per-step distribution	1	0	yes
Top- k truncation	Ψ	per-step support	1	0	yes
Nucleus (top- p) sampling	Ψ	per-step support	1	0	yes
Beam search	Ψ	sequence-level objective	b	0	no
Constrained decoding	Ψ	per-step support	1	0	either
Zero-shot CoT	Ω	reasoning trace	1–2	$O(1)$	no
Few-shot CoT	$\Phi + \Omega$	context and trace	1	$O(n\bar{\ell}_r)$	no
Least-to-most prompting	Ω	trace decomposition	s	$O(s)$	no
Self-consistency	$\Omega + \Psi$	trace ensemble	m	0	yes
Tree-of-thoughts	$\Omega + \Psi$	trace search	$O(bd)$	$O(bd\bar{\ell}_r)$	either
ReAct	$\Phi + \Omega$	trace and context	$O(n_a)$	$O(n_a\bar{\ell}_o)$	yes
Self-refine	$\Phi + \Omega$	trace and context	$2c$	$O(c\bar{\ell}_r)$	no

V. EXPERIMENTAL SETUP

A. Design

We use a fully crossed factorial design with three factors. The algorithmic factor has six levels: direct answering, zero-shot CoT, few-shot CoT with $n = 4$ exemplars, self-consistency with $m = 8$ over few-shot CoT, ToT with $b = 3$ and $d = 3$, and ReAct with a calculator and a retrieval tool. The lexical factor has four levels: four paraphrases of each template, written by different authors and checked to preserve exemplar count, trace shape, and output format, so that the algorithmic content of the four is identical by construction. The decoding factor has eight levels drawn from $T \in \{0.0, 0.3, 0.7, 1.0\}$ crossed with $\{\text{top-}p = 0.9, \text{top-}p = 1.0\}$, with top- k fixed at 40 for $T > 0$. Greedy decoding is used as the reference cell. This design is what permits (11) to be estimated rather than assumed.

B. Models

We evaluate [list the models and parameter counts, e.g. three open-weight instruction-tuned models spanning roughly an order of magnitude in size]. We use open-weight models throughout so that token-level log-probabilities are available; (5) and Proposition 1 both require them, and most hosted APIs do not expose the full distribution. All weights are frozen; no fine-tuning is performed at any stage.

C. Tasks and metrics

Four tasks cover the regimes in which prompting techniques are usually claimed to help: grade-school arithmetic (GSM8K [18]), commonsense multiple choice (CommonsenseQA [19]), program synthesis (HumanEval [20]), and abstractive summarisation. We sample $[n]$ items per task under a fixed random seed and hold the sample constant across every cell. Accuracy is exact match for the first two tasks, pass@1 for the third,

and ROUGE-L for the fourth. Alongside accuracy we record output diversity (distinct-2 and self-BLEU over the samples in a cell), the per-token divergence D_Φ of (5), the realised nucleus width κ , generated tokens, forward passes, and wall-clock latency.

D. Protocol

Every cell is run with $[s]$ seeds. Answers are extracted with a single regular expression shared by all conditions, so that differences in parsing cannot masquerade as differences in reasoning; parse failures are counted as errors and reported separately. Confidence intervals come from a paired bootstrap over items with 10,000 resamples, and comparisons across the algorithmic factor are corrected with the Holm–Bonferroni procedure. Variance components for (11) are estimated by restricted maximum likelihood with items treated as a random effect. Prompts, seeds, raw generations, and analysis scripts are released at [repository URL].

TABLE II
ACCURACY BY ALGORITHMIC LEVEL, AVERAGED OVER FOUR PARAPHRASES

Method	GSM8K	CSQA	HumanEval	Summ.
Direct	[-]	[-]	[-]	[-]
Zero-shot CoT	[-]	[-]	[-]	[-]
Few-shot CoT	[-]	[-]	[-]	[-]
Self-consistency	[-]	[-]	[-]	[-]
Tree-of-thoughts	[-]	[-]	[-]	[-]
ReAct	[-]	[-]	[-]	[-]
Paraphrase range	[±]	[±]	[±]	[±]
AER	[-]	[-]	[-]	[-]

VI. RESULTS AND ANALYSIS

Section VI reports measurements from the design of Section V. Values marked in red must be replaced with the figures produced by your own runs.

A. Algorithmic versus lexical variation

Table II reports accuracy for each algorithmic level, averaged over the four paraphrases and reported at the reference decoding setting, with the spread across paraphrases given as a range. The comparison that bears on **H1** is between the vertical spread of the table—movement between cells of the taxonomy—and the horizontal spread within a row. On [task] the algorithmic factor moves accuracy by [Δ points] while paraphrasing moves it by [δ points], giving AER = [value]; the corresponding figures for the remaining tasks are [summarise]. **H1** is therefore [supported / partially supported / not supported].

B. Interaction with the decoding policy

Table III sweeps temperature and nucleus threshold for three algorithmic levels. Two patterns are of interest. First, the optimal temperature is not shared across methods: greedy CoT peaks at [T], whereas self-consistency requires $T > 0$ by construction and peaks at [T], which is the interaction predicted in **H2**. Second, the accuracy cost of a badly chosen temperature is [magnitude], which is [larger / smaller] than the gap between adjacent methods in Table II. A technique reported at a single undocumented decoding setting is therefore [characterise the resulting ambiguity].

TABLE III
ACCURACY UNDER DECODING SWEEP (GSM8K)

top- p	T	Few-shot CoT	Self-cons.	ToT
1.0	0.0	[-]	n/a	[-]
1.0	0.3	[-]	[-]	[-]
1.0	0.7	[-]	[-]	[-]
1.0	1.0	[-]	[-]	[-]
0.9	0.3	[-]	[-]	[-]
0.9	0.7	[-]	[-]	[-]
0.9	1.0	[-]	[-]	[-]

C. Distributional measurements

Table IV reports the per-token divergence D_Φ between each prompt condition and the direct-answering baseline, the mean realised nucleus width $\bar{\kappa}$, and output diversity. The measurement that matters here is the relationship between D_Φ and accuracy: [state whether the two are monotonically related, and identify any condition with high divergence but no accuracy gain]. A condition of that kind is a prompt that moves the distribution without moving it usefully, which is precisely the case that raw benchmark numbers cannot distinguish from an inert prompt.

TABLE IV
DISTRIBUTIONAL EFFECT RELATIVE TO DIRECT ANSWERING (D_Φ IN NATS/TOKEN)

Condition	D_Φ	$\bar{\kappa}$	dist-2	s-BLEU
Paraphrase (mean)	[-]	[-]	[-]	[-]
Zero-shot CoT	[-]	[-]	[-]	[-]
Few-shot CoT	[-]	[-]	[-]	[-]
Retrieval prompt	[-]	[-]	[-]	[-]

D. Cost

Table V converts accuracy into accuracy per thousand generated tokens. Ranking the methods by raw accuracy and by cost-adjusted accuracy gives [the same / a different] ordering; in particular [name the method whose ranking changes most]. Under a fixed token budget the preferred method is [method], which [does / does not] match the accuracy-optimal choice.

TABLE V
ACCURACY PER THOUSAND GENERATED TOKENS (GSM8K)

Method	Acc.	Tok./item	Acc./1k tok
Direct	[-]	[-]	[-]
Zero-shot CoT	[-]	[-]	[-]
Few-shot CoT	[-]	[-]	[-]
Self-consistency	[-]	[-]	[-]
Tree-of-thoughts	[-]	[-]	[-]
ReAct	[-]	[-]	[-]

E. Error analysis

We hand-label [n] errors per method into four categories: retrieval or knowledge failure, arithmetic slip, invalid reasoning step, and formatting or parse failure. The distribution over categories is [summarise]. Two predictions of Section IV are checkable here. Proposition 3 implies that self-consistency should reduce arithmetic slips, which are high-variance and low-margin, while leaving invalid reasoning steps largely untouched; the observed change is [state]. The framing of ReAct as external conditioning implies that it should reduce knowledge failures specifically; the observed change is [state].

VII. DISCUSSION

The framework in Section IV treats a prompt as a control input, and the experiments are constructed to test whether that view has empirical consequences. Three points deserve emphasis.

Reported gains carry a decoding configuration whether or not it is written down. Because trace-space and decoding-space operators interact, a chain-of-thought result at $T = 0$ and the same result at $T = 0.7$ are different measurements of different quantities. The practical recommendation that follows is narrow and cheap: report T , k , p , and m alongside every prompting result, and report the reference cell against which a gain is measured.

Divergence is necessary but not sufficient. A prompt that leaves $\Delta_t \approx 0$ cannot change model behaviour, so a low D_Φ

is decisive negative evidence about a proposed technique. The converse does not hold. High divergence with no accuracy gain is common, and it identifies prompts that redistribute probability mass without concentrating it on better answers—a failure mode that accuracy alone reports as a null result and that D_Φ reports as a mechanism.

Where a technique’s ceiling lies is determined by the operator class it belongs to. Proposition 3 shows that self-consistency converges to the marginal mode and can do no better, so it cannot rescue an item whose marginal mode is wrong; the fix has to come from prompt space, which changes the distribution, or from trace space with an external signal, as in ToT and ReAct. Reading Table I as a design space rather than a list makes this kind of diagnosis routine: identify which operator the failure implicates, then choose a technique from that class.

A consequence for practitioners is that the search for a better wording is usually the wrong first move. If the algorithmic effect ratio is high, the returns to rewording are bounded and the effort is better spent selecting an operator class and tuning its decoding parameters. If it is low, the reported technique is fragile and should be re-validated on the exact wording that will ship.

VIII. LIMITATIONS AND THREATS TO VALIDITY

Several limitations qualify these conclusions.

Model coverage. We evaluate open-weight models because token-level log-probabilities are required by (5). The largest proprietary models may respond differently, and prompting effects are known to vary with scale; our conclusions should be read as applying to the scales we test.

Paraphrase construction. The lexical factor depends on the claim that our four paraphrases preserve algorithmic structure. We control exemplar count, trace shape, and output format, but a paraphrase can still alter tokenisation boundaries or lexical overlap with pretraining data in ways we do not measure. This makes σ_{lex}^2 in (11) an upper bound on the purely lexical contribution, and hence AER a conservative estimate.

Task coverage. Four tasks cannot represent the space of LLM applications. Open-ended generation, multi-turn dialogue, and agentic workflows with long horizons are absent, and the last of these is where trace-space operators are most heavily used in practice.

Value estimation in ToT. The ToT results depend on a hand-specified value function. A better v would raise the ToT rows in every table, so those numbers are a lower bound on what the method can achieve rather than a measurement of the method itself.

Contamination. Public benchmarks may appear in pre-training corpora. Contamination would inflate absolute accuracies but should affect all cells of the design roughly equally, leaving the contrasts on which **H1** and **H2** depend comparatively robust.

Statistical power. With $[n]$ items per task and $[s]$ seeds, differences smaller than roughly $[e]$ points are not resolvable, and we do not claim them.

IX. CONCLUSION

We have argued that prompt engineering techniques are best analysed as algorithmic operations on the conditional distribution an LLM defines over outputs, and we have given a taxonomy that classifies techniques by whether they act on the prompt, the decoding policy, or the reasoning trace. The formal treatment yields concrete statements: truncation operators bound per-step entropy in ways that differ between top- k and nucleus sampling, greedy chain-of-thought targets the joint mode while self-consistency targets the marginal mode, and the gap between the two accounts for the benefit of voting. A factorial experiment separating lexical from algorithmic variation supplies the corresponding measurements, and the algorithmic effect ratio summarises them in a single number.

The practical recommendation is that prompt engineering results be reported with the decoding configuration, the reference cell, and a divergence measurement attached, so that a reader can tell which operator produced a gain. Future work should extend the framework to multi-turn and agentic settings, where traces are long and the environment supplies part of the conditioning context, and should investigate whether the algorithmic effect ratio predicts how well a prompting technique transfers between models.

ACKNOWLEDGMENT

[Add acknowledgements and funding statement before submission.]

REFERENCES

- [1] T. Brown *et al.*, “Language models are few-shot learners,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.
- [2] S. Min, X. Lyu, A. Holtzman, M. Artetxe, M. Lewis, H. Hajishirzi, and L. Zettlemoyer, “Rethinking the role of demonstrations: What makes in-context learning work?,” in *Proc. Conf. Empir. Methods Nat. Lang. Process. (EMNLP)*, 2022, pp. 10748–10760.
- [3] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 24824–24837.
- [4] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-consistency improves chain of thought reasoning in language models,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.
- [5] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, “Tree of thoughts: Deliberate problem solving with large language models,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 11809–11822.
- [6] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.
- [7] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [8] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, “The curious case of neural text degeneration,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2020.
- [9] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, “Large language models are zero-shot reasoners,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 22199–22213.
- [10] Y. Lu, M. Bartolo, A. Moore, S. Riedel, and P. Stenetorp, “Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity,” in *Proc. Annu. Meeting Assoc. Comput. Linguist. (ACL)*, 2022, pp. 8086–8098.

- [11] Z. Zhao, E. Wallace, S. Feng, D. Klein, and S. Singh, “Calibrate before use: Improving few-shot performance of language models,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2021, pp. 12697–12706.
- [12] A. Madaan *et al.*, “Self-refine: Iterative refinement with self-feedback,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 46534–46594.
- [13] A. Fan, M. Lewis, and Y. Dauphin, “Hierarchical neural story generation,” in *Proc. Annu. Meeting Assoc. Comput. Linguist. (ACL)*, 2018, pp. 889–898.
- [14] C. Meister, T. Pimentel, G. Wiher, and R. Cotterell, “Locally typical sampling,” *Trans. Assoc. Comput. Linguist.*, vol. 11, pp. 102–121, 2023.
- [15] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, “Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing,” *ACM Comput. Surv.*, vol. 55, no. 9, pp. 1–35, 2023.
- [16] S. Schulhoff *et al.*, “The prompt report: A systematic survey of prompting techniques,” 2024, arXiv:2406.06608. [Online]. Available: <https://arxiv.org/abs/2406.06608>
- [17] Y. Zhou, A. I. Muresanu, Z. Han, K. Paster, S. Pitis, H. Chan, and J. Ba, “Large language models are human-level prompt engineers,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.
- [18] K. Cobbe *et al.*, “Training verifiers to solve math word problems,” 2021, arXiv:2110.14168. [Online]. Available: <https://arxiv.org/abs/2110.14168>
- [19] A. Talmor, J. Herzig, N. Lourie, and J. Berant, “CommonsenseQA: A question answering challenge targeting commonsense knowledge,” in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguist. (NAACL)*, 2019, pp. 4149–4158.
- [20] M. Chen *et al.*, “Evaluating large language models trained on code,” 2021, arXiv:2107.03374. [Online]. Available: <https://arxiv.org/abs/2107.03374>